PRO 9

```
class BankAccount {
  int accountNumber;
  String accountHolderName;
  double accountBalance;

  BankAccount(int accountNumber, String accountHolderName, double accountBalance) {
    this.accountNumber = accountNumber;
    this.accountHolderName = accountHolderName;
    this.accountBalance = accountBalance;
  }

  synchronized void deposit(String userName, double amount) {
    if (amount <= 0) {
      System.out.println("Invalid Deposit Amount");
      return;
    }
    accountBalance = accountBalance + amount;
    System.out.println("User Name: " + userName);
    System.out.println("Deposited Amount: ₹" + amount);
    System.out.println("Updated Balance: ₹" + accountBalance);
  }

  synchronized void withdraw(String userName, double amount) {
    if (amount <= 0) {
      System.out.println("Invalid Withdrawal Amount");
      return;
    }
    if (amount <= accountBalance) {
      accountBalance = accountBalance - amount;
      System.out.println("User Name: " + userName);
      System.out.println("Withdrawn Amount: ₹" + amount);
      System.out.println("Updated Balance: ₹" + accountBalance);
    } else {
      System.out.println("Insufficient Balance");
    }
  }

  void displayBalance() {
    System.out.println("Final Account Balance: ₹" + accountBalance);
  }

  public static void main(String[] args) {
    BankAccount account = new BankAccount(1001, "Rahul", 10000);

    UserTransaction user1 = new UserTransaction("User1", account, "Deposit", 2000);
```

```
    UserTransaction user2 = new
UserTransaction("User2", account,
"Deposit", 3000);

    UserTransaction user3 = new
UserTransaction("User3", account,
"Deposit", 1000);

    UserTransaction user4 = new
UserTransaction("User4", account,
"Withdrawal", 4000);

    UserTransaction user5 = new
UserTransaction("User5", account,
"Withdrawal", 3000);


    user1.start();

    user2.start();

    user3.start();

    user4.start();

    user5.start();


    try {
      user1.join();

      user2.join();

      user3.join();

      user4.join();

      user5.join();
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }


    account.displayBalance();
  }
}


class UserTransaction extends Thread {

  String userName;

  BankAccount account;

  String transactionType;

  double transactionAmount;


  UserTransaction(String userName,
BankAccount account,

        String transactionType, double
transactionAmount) {

    this.userName = userName;

    this.account = account;

    this.transactionType =
transactionType;

    this.transactionAmount =
transactionAmount;

  }


  public void run() {

    if
(transactionType.equalsIgnoreCase("Dep
osit")) {

      account.deposit(userName,
transactionAmount);

    } else if
(transactionType.equalsIgnoreCase("Wit
hdrawal")) {

      account.withdraw(userName,
transactionAmount);

    }

  }

}
```

PRO 10

```
import java.util.ArrayList;
import java.util.Scanner;

class Student {
  int id;
  String name;
  String department;
  int semester;

  Student(int id, String name, String department, int semester) {
    this.id = id;
    this.name = name;
    this.department = department;
    this.semester = semester;
  }

  void display() {
    System.out.println("Student ID: " + id);
    System.out.println("Name: " + name);
    System.out.println("Department: " + department);
    System.out.println("Semester: " + semester);
    System.out.println("----------");
  }
}

public class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    ArrayList<Student> students = new ArrayList<>();
    int choice;

    do {
      System.out.println("\n--- Student Record Management ---");
      System.out.println("1. Add Student");
      System.out.println("2. Remove Student");
      System.out.println("3. Search Student");
      System.out.println("4. Display All Students");
      System.out.println("5. Exit");
      System.out.print("Enter your choice: ");
      choice = sc.nextInt();
      sc.nextLine();

      switch (choice) {
        case 1:
          System.out.print("Enter Student ID: ");
          int id = sc.nextInt();
          sc.nextLine();
```

```
            // Check duplicate ID
            boolean exists = false;
            for (Student s : students) {
              if (s.id == id) {
                exists = true;
                break;
              }
            }

            if (exists) {

System.out.println("Student ID already exists.");
            } else {
              System.out.print("Enter Name: ");
              String name = sc.nextLine();
              System.out.print("Enter Department: ");
              String department = sc.nextLine();
              System.out.print("Enter Semester: ");
              int semester = sc.nextInt();
              sc.nextLine();

              Student s = new Student(id, name, department, semester);
              students.add(s);

System.out.println("Student record added successfully.");
            }
            break;

          case 2:
            System.out.print("Enter Student ID to remove: ");
            int removeId = sc.nextInt();
            sc.nextLine();
            boolean removed = false;

            for (int i = 0; i < students.size(); i++) {
              if (students.get(i).id == removeId) {
                students.remove(i);
                removed = true;
                break;
              }
            }

            if (removed) {

System.out.println("Student record removed successfully.");
            } else {

System.out.println("Student record not found.");
            }
```

```
            break;


        case 3:
            System.out.println("1. Search by ID");
            System.out.println("2. Search by Name");
            System.out.print("Enter search option: ");
            int searchChoice = sc.nextInt();
            sc.nextLine();
            boolean found = false;

            if (searchChoice == 1) {
                System.out.print("Enter Student ID: ");
                int searchId = sc.nextInt();
                sc.nextLine();
                for (Student s : students) {
                    if (s.id == searchId) {
                        s.display();
                        found = true;
                        break;
                    }
                }
            } else if (searchChoice == 2) {
                System.out.print("Enter Student Name: ");
                String searchName = sc.nextLine();
                for (Student s : students) {
                    if (s.name.equalsIgnoreCase(searchName)) {
                        s.display();
                        found = true;
                    }
                }
            } else {
                System.out.println("Invalid search option.");
                break;
            }

            if (!found) {
                System.out.println("Student record not found.");
            }
            break;


        case 4:
            if (students.isEmpty()) {
                System.out.println("No student records available.");
            } else {
                for (Student s : students) {
                    s.display();
                }
            }
            break;
```

```
            case 5:
                System.out.println("Exiting
Student Record Management
System.");
                break;


            default:
                System.out.println("Invalid
choice. Please try again.");
        }
    } while (choice != 5);


    sc.close();
  }
}
```